

1)

GEMINI

 **Análise do Erro de Lógica**No seu código, você está sempre calculando o resto a partir da variável original `segundostotais`. Isso faz com que os cálculos não sejam cumulativos. O que está acontecendo: Você calcula os dias corretamente. Ao calcular as horas, minutos e segundos, você volta a olhar para o total inicial. Exemplo: Se o usuário digitar 3661 segundos, seu código diria que são 1 hora e 61 minutos (pois $3661 / 60 = 61$), em vez de 1 hora, 1 minuto e 1 segundo.  **Versão Corrigida**Para funcionar, cada passo deve usar o resto da operação anterior. Veja como ficaria a lógica correta: `Plaintext`INICIO

```
segundostotais = inteiro
```

```
escreva "Digite o tempo total em segundos: "
```

```
leia segundostotais
```

```
// 1. Calcula os dias e o que SOBROU dos dias
```

```
dias = segundostotais / 86400
```

```
resto = segundostotais mod 86400
```

```
// 2. Do que sobrou, calcula as horas e o novo RESTO
```

```
horas = resto / 3600
```

```
resto = resto mod 3600
```

```
// 3. Do que sobrou das horas, calcula os minutos e o resto final
```

```
minutos = resto / 60
```

```
segundos = resto mod 60
```

```
escreva dias, " dias, ", horas, " horas, ", minutos, " minutos e ", segundos, " segundos."
```

FIM

 **Principais Mudanças Realizadas:** Encadeamento do Resto: Note que a variável `horas` agora é calculada dividindo o resto (o que sobrou dos dias), e não o total inicial. Hierarquia de Tempo: O segredo da conversão de unidades é "ir descendo a escada":
 $1 \text{ dia} = 86.400 \text{ s}$
 $1 \text{ hora} = 3.600 \text{ s}$
 $1 \text{ minuto} = 60 \text{ s}$
Limpeza Final: A variável `segundos` recebe diretamente o último resto da divisão por 60.

CHATGPT

1) Objetivo do algoritmo

O algoritmo quer transformar segundos totais em:

dias

horas

minutos

segundos

Sabendo que:

1 dia = 86400 segundos

1 hora = 3600 segundos

1 minuto = 60 segundos

⚠ Problemas no pseudocódigo

❌ 1. Uso errado do valor original

Você está sempre usando segundostotais para calcular tudo.

Exemplo:

horas = segundostotais / 3600

Isso está errado porque as horas devem ser calculadas com o resto dos dias, não com o total original.

❌ 2. O resto está sendo sobrescrito

Você calcula:

resto = segundostotais mod 86400

mas depois ignora ele e usa novamente segundostotais.

O correto seria usar o resto anterior.

❌ 3. Formatação do escreva

Esta linha tem erro de sintaxe:

escreva; dias(:), horas (:), minutos(:), segundos.

Deveria ser algo como:

escreva dias, " dias ", horas, ":", minutos, ":", segundos

✅ Forma correta do algoritmo

INICIO

segundostotais : inteiro

escreva "tempo total em segundos: "

leia segundostotais

```
dias = segundostotais / 86400
resto = segundostotais mod 86400
```

```
horas = resto / 3600
resto = resto mod 3600
```

```
minutos = resto / 60
segundos = resto mod 60
```

```
escreva dias, " dias ", horas, ":", minutos, ":", segundos
```

FIM

2)

GEMINI

Este pseudocódigo descreve o funcionamento de uma Ponte de Wheatstone, um circuito clássico da física e engenharia elétrica usado para medir resistências desconhecidas. No entanto, o código possui erros graves de sintaxe e lógica que impediriam sua execução. Vou analisá-los ponto a ponto e depois apresentar a solução. 🔍 **Análise Crítica**

1. Erros de Sintaxe e Estrutura

- Comando leia usado como atribuição:** O comando `leia` serve para receber dados do teclado. Você o utilizou para tentar fazer um cálculo: `leia rx = (r2 * r3) / r1`. O correto seria `rx = (r2 * r3) / r1`.
- Condição órfão:** Existe um `senão`, mas não existe o comando `se` (`if`) para abrir a condição.
- Atribuição em comparação:** Em programação, para comparar se um lado é igual ao outro, usamos `==`. O sinal `=` sozinho serve para guardar um valor em uma variável.
- Redeclaração:** Você declarou `rx = real` no meio do código, o que já havia sido feito no início.

2. Falha na Lógica de Equilíbrio

Uma ponte de Wheatstone está equilibrada quando o produto das resistências opostas é igual: $R_1 \cdot R_x = R_2 \cdot R_3$. No seu código, você primeiro calcula `R_x` para que ele seja o valor de equilíbrio e depois pergunta se está equilibrado. Da forma como está, o resultado seria sempre "equilibrada", pois você forçou o valor de `R_x` no cálculo anterior. Para o algoritmo fazer sentido, o usuário deve fornecer os quatro valores (`R_1`, `R_2`, `R_3`, `R_x`) e o programa deve verificar se a proporção é válida. 🛠️

Pseudocódigo Corrigido (Lógica de Verificação)

Aqui está a forma correta de escrever esse pensamento: Wheatstone bridge, also known as the resistance bridge, calculates the unknown resistance by balancing two legs of the bridge circuit.

Plaintext

```
r1, r2, r3, rx: real
```

```
escreva "Digite os valores das quatro resistências (R1, R2, R3 e Rx):"
```

```
leia r1, r2, r3, rx
```

```
// Verifica a condição de equilíbrio: R1 * Rx = R2 * R3
```

```
SE (r1 * rx) == (r2 * r3) ENTÃO
```

```
    escreva "A ponte está equilibrada."
```

SENÃO

escreva "A ponte não está equilibrada."

// Opcional: calcular qual seria o Rx ideal para equilibrar

$ideal = (r2 * r3) / r1$

escreva "Para equilibrar, Rx deveria ser: ", ideal

FIM_SE

FIM

🐍 Implementação em Python Se você quiser testar essa lógica na prática, aqui está o código equivalente: Python# Entrada de dados

$r1 = \text{float}(\text{input}(\text{"Valor de R1: "}))$

$r2 = \text{float}(\text{input}(\text{"Valor de R2: "}))$

$r3 = \text{float}(\text{input}(\text{"Valor de R3: "}))$

$rx = \text{float}(\text{input}(\text{"Valor de Rx: "}))$

Verificação da condição de equilíbrio

Usamos round() para evitar problemas de precisão com números decimais (float)

if $(r1 * rx) == (r2 * r3)$:

 print("\nA ponte está equilibrada!")

else:

 print("\nA ponte não está equilibrada.")

$ideal = (r2 * r3) / r1$

 print(f"O valor de Rx para equilibrar a ponte deveria ser {ideal}")

código python

Algoritmo de Verificação da Ponte de Wheatstone

print("--- Verificador de Equilíbrio de Ponte ---")

Entrada de dados (convertendo para float para aceitar casas decimais)

$r1 = \text{float}(\text{input}(\text{"Digite o valor de R1: "}))$

$r2 = \text{float}(\text{input}(\text{"Digite o valor de R2: "}))$

$r3 = \text{float}(\text{input}(\text{"Digite o valor de R3: "}))$

$rx = \text{float}(\text{input}(\text{"Digite o valor de Rx (resistência a testar): "}))$

Cálculo dos produtos cruzados

produto_conhecido = $r1 * rx$

produto_oposto = $r2 * r3$

Verificação da condição de equilíbrio: $R1 * Rx = R2 * R3$

Usamos round() para evitar erros de precisão decimal inerentes ao computador

if $\text{round}(\text{produto_conhecido}, 4) == \text{round}(\text{produto_oposto}, 4)$:

 print("\n[RESULTADO]: A ponte está equilibrada!")

else:

```
print("\n[RESULTADO]: A ponte NÃO está equilibrada.")

# Cálculo do valor ideal para o equilíbrio
if r1 != 0:
    rx_ideal = (r2 * r3) / r1
    print(f"Para equilibrar o circuito, Rx deveria ser: {rx_ideal:.2f} Ohms")
else:
    print("Erro: R1 não pode ser zero para o cálculo de equilíbrio.")
```

CHATGPT

1 Declaração das variáveis

Você escreveu:

```
r1 = real
r2 = real
r3 = real
rx = real
```



Problema:

Isso parece atribuição, mas na verdade você quer declarar o tipo da variável.

Forma mais correta:

```
r1, r2, r3, rx : real
```

2 Falta de leitura das variáveis

Você escreveu:

```
escreva "valor de r1"
escreva "valor de r2"
escreva "valor de r3"
```

Mas não leu os valores.

O correto seria:

```
escreva "valor de r1"
leia r1
```

```
escreva "valor de r2"
leia r2
```

escreva "valor de r3"

leia r3

③ Erro grave no cálculo de Rx

Você escreveu:

leia rx = (r2 * r3) / r1

⚠ Problema:

leia serve apenas para entrada de dados
cálculo deve ser feito separado.

Correto:

rx = (r2 * r3) / r1

④ Declaração repetida

Você colocou depois:

rx = real

⚠ Isso não pode acontecer porque rx já foi declarado antes.

⑤ Condição escrita de forma incorreta

Você escreveu:

leia (rx * r1) = (r2 * r3)

⚠ Problema:

leia não é usado em comparação
falta o SE

O correto seria:

se (rx * r1) = (r2 * r3) então

⑥ Estrutura condicional incompleta

Faltou fimse.

✅ Pseudocódigo corrigido

INICIO

r1, r2, r3, rx : real

escreva "valor de r1:"

leia r1

escreva "valor de r2:"

leia r2

escreva "valor de r3:"

leia r3

$rx = (r2 * r3) / r1$

escreva "valor de rx = ", rx

se $(rx * r1) = (r2 * r3)$ então

 escreva "a ponte esta equilibrada"

senão

 escreva "a ponte nao esta equilibrada"

fimse

FIM



Resumo dos erros

Erro Motivo

r1 = real declaração incorreta

faltou leia r1 r2 r3 valores nunca são lidos

leia rx = ... cálculo não usa leia

rx = real repetido variável já declarada

leia $(rx*r1)$ comparação escrita errado

faltou fimse estrutura incompleta



Observação importante:

Se você calcula rx usando a fórmula da ponte, a condição sempre será verdadeira, porque ela vem da mesma equação.



Se quiser, posso também te mostrar algo que muitos professores cobram em prova:

uma versão correta e mais realista do algoritmo da ponte de Wheatstone

um pseudocódigo nota 10 para prova

como esse exercício vira uma pegadinha em lógica de programação.

escreva o código em python do pseudocódigo já corrigido

Aqui está o código em Python baseado no pseudocódigo corrigido. Ele lê os valores dos resistores, calcula Rx e verifica se a ponte está equilibrada. ⚡💻

```
# leitura dos valores dos resistores
r1 = float(input("Valor de R1: "))
r2 = float(input("Valor de R2: "))
r3 = float(input("Valor de R3: "))

# cálculo do resistor desconhecido
rx = (r2 * r3) / r1

print("Valor de Rx =", rx)

# verificação do equilíbrio da ponte
if (rx * r1) == (r2 * r3):
    print("A ponte está equilibrada")
else:
    print("A ponte não está equilibrada")
```

🔍 Como o código funciona

1) Entrada de dados

```
r1 = float(input("Valor de R1: "))
```

input() lê o valor digitado
float() permite números decimais

2) Cálculo de Rx

Rx=
R1
R2×R3

```
rx = (r2 * r3) / r1
```

3) Verificação da ponte

A ponte de Wheatstone está equilibrada quando:

$R1 \times Rx = R2 \times R3$
if (rx * r1) == (r2 * r3):

✓ Exemplo de execução

Valor de R1: 100
Valor de R2: 200
Valor de R3: 300
Valor de Rx = 600.0
A ponte está equilibrada

Código em Python.

```
# leitura dos valores dos resistores
r1 = float(input("Valor de R1: "))
r2 = float(input("Valor de R2: "))
r3 = float(input("Valor de R3: "))

# cálculo do resistor desconhecido
rx = (r2 * r3) / r1

print("Valor de Rx =", rx)

# verificação do equilíbrio da ponte
if (rx * r1) == (r2 * r3):
    print("A ponte está equilibrada")
else:
    print("A ponte não está equilibrada")
```

3)

GEMINI

Este pseudocódigo descreve um cálculo tributário em cascata (por "dentro"), onde um imposto acaba compondo a base de cálculo do próximo. É um modelo comum na contabilidade brasileira. 🔍 **Análise do Pseudocódigo** Lógica em Cascata: O código está logicamente bem estruturado para o que se propõe. Ele segue a sequência: $IPI \rightarrow ICMS \rightarrow PIS/COFINS$. Variável não inicializada: Há um problema no leia pis_cofins. No seu pseudocódigo, você declarou a variável, mas esqueceu de pedir para o usuário digitá-la. Sem esse valor, o cálculo do PIS/COFINS resultará em zero ou erro. Diferenciação de Maiúsculas/Minúsculas: Note que você declarou icms (minúsculo), mas usou ICMS (maiúsculo) no leia. Em programação, isso pode causar erros de "variável não definida". 🐞 **Versão em Python** No Python, usaremos float para lidar com os valores monetários e organizaremos a saída para que os valores fiquem legíveis (com duas casas decimais). Python print("--- Simulador de Impostos em Cascata ---")

```
# Entrada de dados
preco_fabrica = float(input("Digite o preço de fábrica do produto: "))
ipi_perc = float(input("Digite a porcentagem do IPI: "))
```

```
icms_perc = float(input("Digite a porcentagem do ICMS: "))
pis_cofins_perc = float(input("Digite a porcentagem do PIS/COFINS: ")) # Adicionado entrada faltante
```

```
# 1. Cálculo do IPI (Sobre o preço de fábrica)
```

```
valor_ipi = preco_fabrica * (ipi_perc / 100)
```

```
# 2. Cálculo do ICMS (Base = Fábrica + IPI)
```

```
base_icms = preco_fabrica + valor_ipi
```

```
valor_icms = base_icms * (icms_perc / 100)
```

```
# 3. Cálculo do PIS/COFINS (Base = ICMS + sua base anterior)
```

```
base_pis_cofins = base_icms + valor_icms
```

```
valor_pis_cofins = base_pis_cofins * (pis_cofins_perc / 100)
```

```
# 4. Valor Final
```

```
valor_final = base_pis_cofins + valor_pis_cofins
```

```
# Exibição dos resultados formatados
```

```
print("-" * 40)
```

```
print(f"Preço de Fábrica:      R$ {preco_fabrica:>10.2f}")
```

```
print(f"IPI Calculado:      R$ {valor_ipi:>10.2f}")
```

```
print(f"Base para ICMS:      R$ {base_icms:>10.2f}")
```

```
print(f"ICMS Calculado:      R$ {valor_icms:>10.2f}")
```

```
print(f"Base para PIS/COFINS: R$ {base_pis_cofins:>10.2f}")
```

```
print(f"PIS/COFINS Calculado: R$ {valor_pis_cofins:>10.2f}")
```

```
print("-" * 40)
```

```
print(f"VALOR FINAL DO PRODUTO: R$ {valor_final:>10.2f}")
```

💡 O que você deve notar nesta versão: Entrada faltante: Eu adicionei o input para o PIS/COFINS, caso contrário o programa não saberia qual alíquota aplicar. Formatação de Moeda: Usei o comando `>10.2f` dentro das f-strings. Isso faz com que o número tenha sempre 2 casas decimais e fique alinhado à direita, facilitando a leitura da tabela. Ordem de Precedência: O uso de parênteses em `(ipi_perc / 100)` não é estritamente necessário pela regra matemática, mas ajuda muito na leitura do código, deixando claro que você está convertendo a porcentagem em um fator decimal primeiro.

CHATGPT

🔍 Análise do pseudocódigo

O algoritmo calcula o valor final de um produto industrial considerando três tributos aplicados em sequência.

Ordem dos cálculos

❶ IPI

Aplicado sobre o preço de fábrica

$\text{valor_ipi} = \text{preco_fabrica} * \text{ipi} / 100$

❷ ICMS

Aplicado sobre:

$\text{preco_fabrica} + \text{valor_ipi}$

❸ PIS/COFINS

Aplicado sobre:

$\text{base_icms} + \text{valor_icms}$

❹ Valor final

$\text{valor_final} = \text{base_pis_cofins} + \text{valor_pis_cofins}$

Depois o algoritmo exibe um demonstrativo com cada etapa.

⚠ Pequenos problemas no pseudocódigo

❶ Variável escrita diferente:

leia ICMS

mas a variável declarada foi

icms

❷ pis_cofins não é lido no algoritmo.

❸ Existe uma barra desnecessária:

$\text{preco_fabrica} \backslash$

Isso não é necessário.



Versão em Python

Cálculo de tributos de um produto

```

preco_fabrica = float(input("Digite o preço de fábrica do produto: "))
ipi = float(input("Digite a porcentagem do IPI: "))
icms = float(input("Digite a porcentagem do ICMS: "))
pis_cofins = float(input("Digite a porcentagem do PIS/COFINS: "))

# cálculo do IPI
valor_ipi = preco_fabrica * ipi / 100

# cálculo do ICMS
base_icms = preco_fabrica + valor_ipi
valor_icms = base_icms * icms / 100

# cálculo do PIS/COFINS
base_pis_cofins = base_icms + valor_icms
valor_pis_cofins = base_pis_cofins * pis_cofins / 100

# valor final
valor_final = base_pis_cofins + valor_pis_cofins

# demonstrativo
print("\n--- DEMONSTRATIVO ---")
print("Preço de fábrica:", preco_fabrica)
print("IPI calculado:", valor_ipi)
print("Base para ICMS:", base_icms)
print("ICMS calculado:", valor_icms)
print("Base para PIS/COFINS:", base_pis_cofins)
print("PIS/COFINS calculado:", valor_pis_cofins)
print("Valor final do produto:", valor_final)

```

4)

GEMINI

Análise Técnica do Pseudocódigo

Lógica de Hierarquia: O código funciona como um "funil". Ele começa pelos casos mais críticos (Inapto), passa pelos intermediários (Restrito), verifica o cenário ideal (Excelente) e termina no padrão (Adequado).

Condicionais Aninhados: Dentro da categoria "RESTRITO", o código detalha qual parâmetro falhou, o que é muito útil para um laudo técnico.

Operadores Lógicos:

Usa-se o OU (ou) para situações de risco (se um dos dois falhar, o laudo é prejudicado).

Usa-se o E (e) para a excelência (ambos precisam ser altos simultaneamente).

Versão em Python

No Python, substituímos o senão se pelo comando elif e os blocos finse pela indentação (o recuo do texto).

Python

Sistema de Análise Geotécnica de Solo

```
print("--- Análise de Solo para Fundações ---")
```

Entrada de dados

```
spt = int(input("Valor do SPT (golpes): "))
```

```
cap_carga = float(input("Capacidade de carga (kPa): "))
```

```
print("-" * 30)
```

Verificação das condições

```
if spt < 5 or cap_carga < 80:
```

```
    print("LAUDO: INAPTO")
```

```
    print("Solo muito fraco — reforço necessário.")
```

```
elif spt < 10 or cap_carga < 150:
```

```
    print("LAUDO: RESTRITO")
```

```
    print("Fundação especial recomendada.")
```

Detalhamento do limite

```
if spt < 10:
```

```
    print("-> Parâmetro no limite: SPT baixo.")
```

```
if cap_carga < 150:
```

```
    print("-> Parâmetro no limite: capacidade de carga baixa.")
```

```
elif spt >= 30 and cap_carga >= 300:
```

```
    print("LAUDO: EXCELENTE")
```

```
    print("Solo de alta capacidade.")
```

```
else:
```

```
    print("LAUDO: ADEQUADO")
```

```
    print("Fundação convencional viável.")
```

```
print("-" * 30)
```

 O que observar na implementação:

Identação: Em Python, não existe FIMSE. O que define o que está dentro do if ou elif é o

espaço (recuo) no início da linha. Se você tirar o recuo, o Python entende que o bloco terminou.

Eficiência do elif: O programa testa as condições em ordem. Se ele cair no primeiro if (Inapto), ele ignora todos os elif abaixo, economizando processamento.

Flexibilidade: Note que no caso "Restrito", usei dois if separados para os avisos de parâmetro no limite. Isso permite que o programa exiba ambas as mensagens caso tanto o SPT quanto a carga estejam baixos, enriquecendo o laudo.

CHATGPT

 Análise do pseudocódigo

O algoritmo gera um laudo de viabilidade de fundação baseado em dois parâmetros do solo:

SPT → resistência do solo (número de golpes)

cap_carga → capacidade de carga do solo em kPa

 Lógica usada

① Solo muito fraco

spt < 5 OU cap_carga < 80

Resultado:

INAPTO

② Solo com restrições

spt < 10 OU cap_carga < 150

Resultado:

RESTRITO

Além disso, o algoritmo identifica qual parâmetro está no limite.

③ Solo excelente

spt >= 30 E cap_carga >= 300

Resultado:

EXCELENTE

④ Caso intermediário

Se nenhuma condição anterior ocorrer:

ADEQUADO

Versão em Python

Sistema de classificação de solo para fundação

```
spt = int(input("Valor do SPT: "))
cap_carga = float(input("Digite a capacidade de carga (kPa): "))

if spt < 5 or cap_carga < 80:
    print("LAUDO: INAPTO")
    print("Solo muito fraco — reforço necessário")

elif spt < 10 or cap_carga < 150:
    print("LAUDO: RESTRITO")
    print("Fundação especial recomendada")

if spt < 10:
    print("Parâmetro no limite: SPT baixo")

if cap_carga < 150:
    print("Parâmetro no limite: capacidade de carga baixa")

elif spt >= 30 and cap_carga >= 300:
    print("LAUDO: EXCELENTE")
    print("Solo de alta capacidade")

else:
    print("LAUDO: ADEQUADO")
    print("Fundação convencional viável")
```

Exemplo de execução

Entrada:

SPT = 15
cap_carga = 120

Saída:

LAUDO: RESTRITO
Fundação especial recomendada
Parâmetro no limite: capacidade de carga baixa

✓ O código Python mantém exatamente a mesma lógica do pseudocódigo, usando:

or → equivalente ao OU

and → equivalente ao E
elif → equivalente ao SENÃO SE

5)

GEMINI

Análise Crítica do Pseudocódigo Erro de Lógica na Parte A: A condição $\text{se desvio} \geq \text{tolerancia_abs}$ e $\text{desvio} \leq \text{tolerancia_abs}$ é matematicamente impossível de ser útil, pois só seria verdadeira se o desvio fosse exatamente igual à tolerância. Além disso, ela ignora desvios negativos. Correção: Em engenharia, usamos o módulo (valor absoluto) para garantir que o desvio, seja para mais ou para menos, não ultrapasse o limite. Variáveis não declaradas: No pseudocódigo, as variáveis são declaradas no início de cada parte, o que é uma boa prática de organização. Lógica da Parte B: Está correta e utiliza uma margem de segurança de 10% (fator 1.10 e 0.90). 🐍 Versão em Python No Python, utilizei a função `abs()` para corrigir a lógica da solda e formatei as saídas para facilitar a leitura técnica. Python

```
# --- PARTE A:
CONTROLE DE QUALIDADE DE SOLDA ---
print(">>> ANÁLISE DE RESISTÊNCIA DA SOLDA <<<")
```

```
resistencia_nominal = float(input("Resistência nominal (MPa): "))
resistencia_medida = float(input("Resistência medida (MPa): "))
```

```
# Cálculo da tolerância (8% conforme o pseudocódigo)
tolerancia_abs = resistencia_nominal * 0.08
desvio = resistencia_medida - resistencia_nominal
desvio_pct = (desvio / resistencia_nominal) * 100
```

```
# Lógica corrigida: o valor absoluto do desvio deve ser menor ou igual à tolerância
if abs(desvio) <= tolerancia_abs:
    print("LAUDO SOLDA: SOLDA APROVADA")
else:
    print("LAUDO SOLDA: SOLDA REPROVADA")
```

```
print(f"Desvio: {desvio:.2f} MPa")
print(f"Desvio percentual: {desvio_pct:.2f}%")
print("-" * 40)
```

```
# --- PARTE B: MONITORAMENTO DA ESTEIRA ---
print("\n>>> MONITORAMENTO DA ESTEIRA <<<")
```

```
velocidade_alvo = float(input("Velocidade alvo (m/min): "))
velocidade_atual = float(input("Velocidade atual (m/min): "))
```

```
# Verificação de faixas de velocidade (+/- 10%)
```



```

if velocidade_atual > velocidade_alvo * 1.10:
    print("LAUDO ESTEIRA: VELOCIDADE ALTA — risco de posicionamento incorreto")
elif velocidade_atual < velocidade_alvo * 0.90:
    print("LAUDO ESTEIRA: VELOCIDADE BAIXA — gargalo na produção")
else:
    print("LAUDO ESTEIRA: VELOCIDADE OK")

```

🔴 O que foi melhorado na tradução: A Função abs(): No teste da solda, usei if abs(desvio) <= tolerancia_abs. Isso verifica se a solda está dentro da margem de erro, não importa se ela é um pouco mais forte ou um pouco mais fraca que o nominal. F-Strings com Precisão: Usei {desvio:.2f} para garantir que o relatório mostre apenas duas casas decimais, evitando números como 0.30000000004. Uso do elif: Na Parte B, o elif substitui o "senão se" de forma mais elegante, garantindo que apenas um laudo seja emitido por vez. Você percebeu como a correção do abs() mudou a viabilidade do laudo da solda? Sem isso, quase todas as soldas seriam reprovadas!

CHATGPT

analisando seu pseudocódigo 🔍:

Há um pequeno erro lógico na condição da Parte A:

se desvio >= tolerancia_abs e desvio <= tolerancia_abs

Assim o valor teria que ser ao mesmo tempo maior ou igual e menor ou igual ao mesmo número, o que só seria possível se fosse exatamente igual.

O correto seria verificar o intervalo $\pm$ tolerância:

se desvio >= -tolerancia_abs e desvio <= tolerancia_abs

Versão em Python 🐍

```

# =====
# PARTE A - INSPEÇÃO DE SOLDA
# =====

```

```
resistencia_nominal = float(input("Escreva a resistência nominal (MPa): "))
```

```
resistencia_medida = float(input("Escreva a resistência medida (MPa): "))
```

```
tolerancia_abs = resistencia_nominal * 0.08
```

```
desvio = resistencia_medida - resistencia_nominal
```

```
desvio_pct = (desvio / resistencia_nominal) * 100
```

```
if desvio >= -tolerancia_abs and desvio <= tolerancia_abs:
```

```

    print("LAUDO SOLDA: SOLDA APROVADA")
else:
    print("LAUDO SOLDA: SOLDA REPROVADA")
    print("Desvio:", desvio, "MPa")
    print("Desvio percentual:", desvio_pct, "%")

# =====
# PARTE B - VELOCIDADE DA ESTEIRA
# =====

velocidade_alvo = float(input("Digite a velocidade alvo (m/min): "))
velocidade_atual = float(input("Digite a velocidade atual (m/min): "))

if velocidade_atual > velocidade_alvo * 1.10:
    print("LAUDO ESTEIRA: VELOCIDADE ALTA — risco de posicionamento incorreto")

elif velocidade_atual < velocidade_alvo * 0.90:
    print("LAUDO ESTEIRA: VELOCIDADE BAIXA — gargalo na produção")

else:
    print("LAUDO ESTEIRA: VELOCIDADE OK")

```

Como executar 

❶ Salve o arquivo como:

inspecao_robo.py

❷ Abra o CMD ou terminal na pasta.

❸ Execute:

python inspecao_robo.py

ANALIZE DAS LLMS

embora as duas llms tenham analisado e resolvido os problemas dos pseudocódigos entregue para elas, ainda assim o Gemini se mostrou mais completo tanto em explicações de solução de erros quanto na criação do código python aparentemente mais completo.